

Designing a State-of-the-Art Autonomous AI-First iPhone Device Lab for Testing, Debugging, Security Research, and CI at Scale

Research date: 17 September 2026

Target environment: closed/on-premises R&D organisation

Primary objective: maximum technically achievable control, observability, debugging, instrumentation, and autonomous testing of heterogeneous physical iPhones, without pretending that stock iOS provides Android/ADB-level privilege.

Executive findings and design principles

The central conclusion is straightforward:

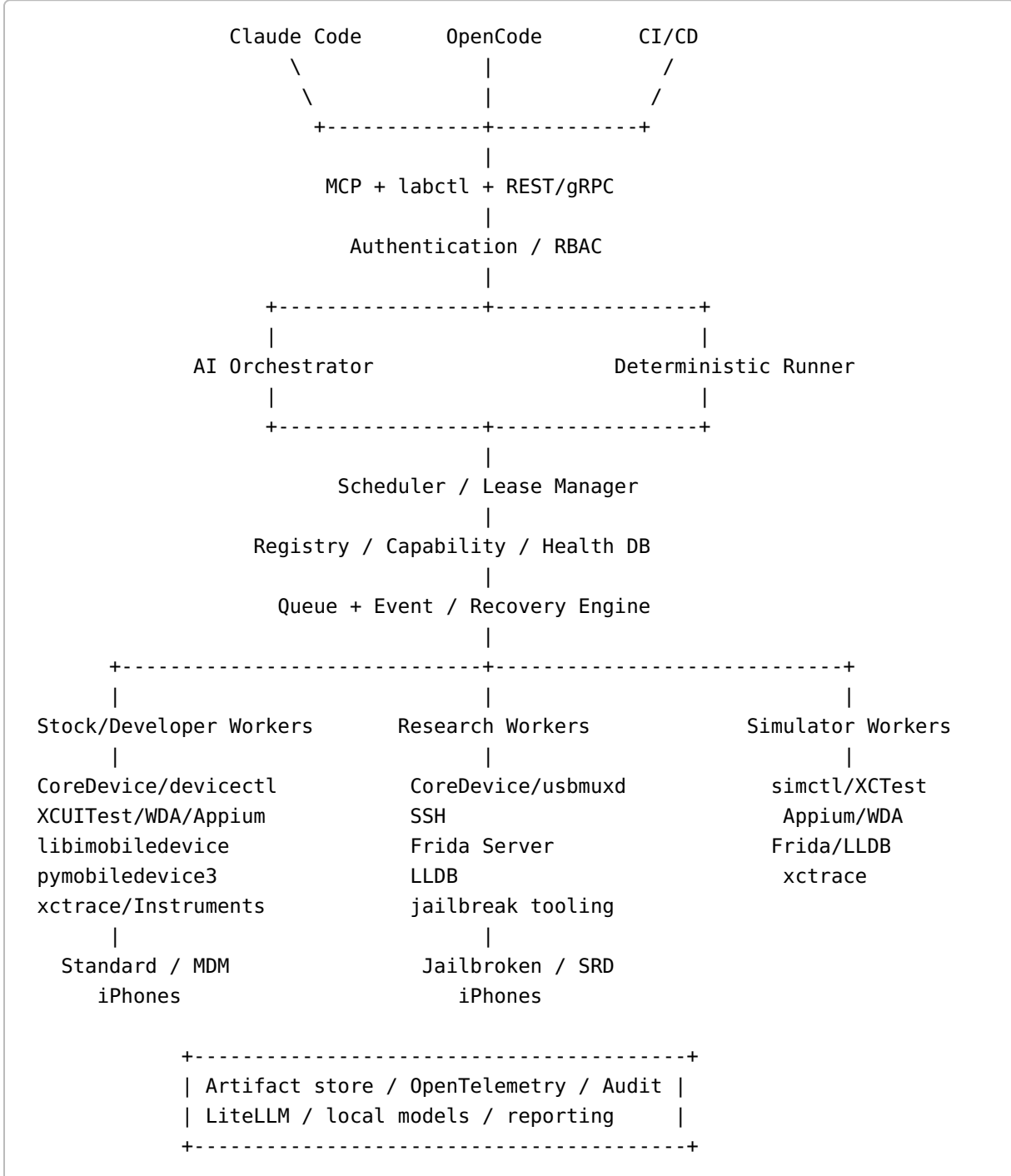
There is no single “ADB for iPhone”, and no existing open-source project provides the complete platform described in this assignment. The correct architecture is a capability-negotiating internal control plane that composes Apple-native tooling, WebDriverAgent/XCUITest, CoreDevice/devicectl, libimobiledevice/pymobiledevice3, MDM where useful, Frida/LLDB, and a deliberately isolated jailbroken/Security Research Device tier.

Apple’s security model deliberately separates UI testing, application development, device management, debugging, filesystem access, process access, and privileged runtime instrumentation. Developer Mode enables development workflows; it does **not** grant a shell, root, broad filesystem access, or arbitrary process debugging. Supervision/MDM adds administrative commands and configuration controls; it likewise does **not** grant Unix-shell access. XCUITest/WebDriverAgent gives very substantial UI control, but is not a system-management or instrumentation API. The missing capabilities have to come from jailbreaks, an Apple Security Research Device, application-specific instrumentation such as Frida Gadget, or simulator-only facilities. Apple introduced Developer Mode in iOS 16 specifically as a requirement for development-signed code and development workflows, while preserving the rest of the platform’s security model. ¹

The current Apple toolchain has also moved further toward **CoreDevice/devicectl** and **RemoteXPC**. Apple documents `devicectl` as the command-line mechanism to manage and interact with devices attached to a host. Appium’s 2026 documentation now explicitly distinguishes newer iOS versions that rely on RemoteXPC tunnels, and its iOS 27 handling no longer assumes the older `devicectl` fallback can start a preinstalled XCTest runner. ²

As of this research date, Apple’s developer materials are on the Xcode 27/iOS 27 generation, so any new laboratory architecture should be designed around the post-iOS-17 CoreDevice/RemoteXPC world rather than older `idevice*`-only assumptions. ³

Recommended architecture in one view



The important architectural distinction is that **MCP should be an interface to the lab, not the lab itself**. MCP is an open protocol for connecting AI applications to tools and data; it says nothing about device leases, fencing, scheduling, recovery, USB ownership, signing state, health, or production safety. ⁴

Likewise, **Appium should be an adapter, not the global control plane**. Appium's current XCUITest driver is a mature way to expose WebDriver semantics over XCTest/WDA, and its parallel-session architecture already accounts for per-device UDIDs, WDA ports, MJPEG ports, and derived-data paths. Those mechanics belong inside each Mac worker/session manager. ⁵

Evidence classification used in this report

Label	Meaning
V — Verified	Official Apple/project documentation or current primary repository confirms the capability.
E — Experimental/ documented	Implemented/documented upstream but with substantial restrictions or unstable/private protocol dependencies.
C — Community- reported	Community tooling or reports without comparable vendor guarantees.
R — Research prototype	Academic or early-stage agent/research implementation.
T — Theoretical	Architecturally plausible, but no credible current implementation evidence.
U — Unsupported/ unverified	No supported mechanism or evidence sufficient for production claims.

The most important “do not confuse these” map

Requirement	Primary technology	What it does not give you
UI automation	XCUITest / WDA / Appium	Shell, root filesystem, arbitrary memory inspection
App lifecycle	CoreDevice/devicectl, XCTest/WDA	General device administration
Administrative management	MDM/supervision	Shell, root, arbitrary process control
Apple device services	CoreDevice, libimobiledevice, pymobiledevice3	ADB-equivalent Unix shell
Debugging	LLDB/Xcode	Arbitrary stock system-process attach
Profiling	Instruments/xctrace	Root filesystem or runtime hooking
Application instrumentation	Frida Gadget / debugger	System-wide instrumentation on stock iOS
System-wide instrumentation	Jailbreak + Frida Server, or SRD	Availability on arbitrary current hardware/iOS

Requirement	Primary technology	What it does not give you
Broad filesystem/ process/shell	Jailbreak/SRD	Guaranteed compatibility with every iPhone/iOS
CI scheduling	Internal scheduler	Supplied by none of the above
Autonomous reasoning	AI orchestrator	Deterministic correctness or privilege by itself

This separation should be treated as a foundational design rule.

Capability reality on iOS

The six device classes

Standard physical iPhone means a normally paired, non-jailbroken retail iPhone with no assumption that Developer Mode is enabled.

Developer-mode physical iPhone is still stock iOS, but Developer Mode permits development-signed applications, XCTest/WDA workflows and supported debugging of appropriately signed/debuggable applications. Apple explicitly requires Developer Mode for development workflows beginning with iOS 16-class systems. ¹

Supervised/MDM-managed iPhone adds organisational administration. Apple supervision exposes additional restrictions/configuration options, and MDM has commands for operations such as restart, shutdown, lock, erase and related lifecycle management. Supervision is administrative authority, not root access. ⁶

Jailbroken iPhone is the closest practical equivalent to the low-level Android research target. Depending on exploit/jailbreak and OS version, it can offer SSH, a package/bootstrap environment, broad filesystem/process access and Frida Server. Availability is highly model/version specific.

Apple Security Research Device is a special Apple-provided iPhone explicitly intended for security research. Apple says SRDs provide shell access, permit arbitrary research tools, custom entitlements and kernel customisation while retaining security properties intended for meaningful iOS research. Apple Pay and third-party applications are excluded from the SRD programme's normal research scope. ⁷

iOS Simulator offers the broadest host-side programmability, but it is not hardware iOS. It lacks crucial equivalence for Secure Enclave behaviour, many radios, hardware security boundaries, some kernel/device behaviour, actual power/thermal characteristics, and physical-device signing/provisioning paths.

Comprehensive capability matrix

Legend: **S** = supported; **P** = partial/conditional; **U** = unsupported through ordinary mechanisms; **N/A** = concept does not meaningfully apply.

Capability	Stock	Dev Mode	Managed	Jailbroken	SRD	Simulator	Primary mechanism / limitation
Discover device	S	S	S	S	S	S	CoreDevice/ devicectl, usbmuxd/ libimobiledevice; <code>simctl</code> for simulator
Reserve device	S	S	S	S	S	S	Lab scheduler function , not an iOS API
Pair device	S	S	S	S	S	N/A	Lockdown/ CoreDevice trust relationship
USB connectivity	S	S	S	S	S	N/A	Apple MobileDevice/ CoreDevice/ usbmuxd
Network connectivity	P	S	S	S	S	S	Paired wireless development/Wi- Fi sync; SSH for research devices
Automatic reconnect	P	P	P	S	S	S	Worker recovery around usbmuxd/ RemoteXPC/ tunnels
Install IPA	P	S	S	S	S	P	Correct signing/ distribution required; simulator consumes simulator builds, not ordinary device IPA
Uninstall app	S/P	S	S	S	S	S	installation proxy/devicectl/ WDA/MDM

Capability	Stock	Dev Mode	Managed	Jailbroken	SRD	Simulator	Primary mechanism / limitation
Launch app	P	S	P	S	S	S	devicectl/WDA/XCTest; MDM itself is not a generic foreground-app launcher
Terminate app	P	S	P	S	S	S	CoreDevice/WDA; broader process kill requires privilege
Restart app	P	S	P	S	S	S	terminate + launch
Reboot device	P	P	S	S	S	S	diagnostics service/MDM/shell; simulator boot lifecycle
Shutdown device	P	P	S	S	S	S	diagnostics relay/MDM/shell
Lock	P	P	S	S	S	S	MDM/XCTest/device APIs depending context
Unattended passcode unlock	U	U	P	P	P	S	Stock iOS deliberately prevents generic passcode bypass; MDM may clear passcodes under allowed conditions, not “type the secret for us”
Change orientation	U	S	P	S	S	S	XCTest/WDA

Capability	Stock	Dev Mode	Managed	Jailbroken	SRD	Simulator	Primary mechanism / limitation
Screenshot	P	S	P	S	S	S	screenshot services/WDA; <code>simctl</code> on simulator
Record screen	P	S	P	S	S	S	XCTest/Appium/ platform recording/ simulator tooling
Stream screen	U/P	P	P	S	S	S	WDA MJPEG/ screenshot stream; not unrestricted raw framebuffer access
Tap	U	S	P	S	S	S	XCTest/WDA
Swipe	U	S	P	S	S	S	XCTest/WDA
Type	U	S	P	S	S	S	XCTest/WDA
Long press	U	S	P	S	S	S	XCTest/WDA
Multi-touch	U	P	P	P/S	P/S	S	XCTest coordinate actions; framework/ version dependent
Read accessibility/UI hierarchy	U	S	P	S	S	S	XCTest/WDA accessibility snapshot
SpringBoard interaction	U	P	P	S	S	P/S	XCTest can see system UI in many cases, but behaviour is less stable than app-owned UI

Capability	Stock	Dev Mode	Managed	Jailbroken	SRD	Simulator	Primary mechanism / limitation
Settings interaction	U	P	P	S	S	P/S	UI automation, plus MDM for configuration that has a management payload
Permission-dialog interaction	U	S/P	P	S	S	S	XCTest alert handling/WDA
Notification interaction	U	P	P	S	S	P/S	UI automation of Notification Centre/system UI; brittle
Safari automation	P	S	P/S	S	S	S	WebKit remote debugging/ Appium Safari/ XCTest
Arbitrary-app UI automation	U	S/P	P	S	S	S	XCUITest can interact with installed apps at UI level; this does not imply debugging/ instrumentation rights
Query installed applications	S	S	S	S	S	S	installation proxy/ CoreDevice/ pymobiledevice3
Inspect application container	P	S/P	P	S	S	S	House Arrest/ AFC for eligible apps; own debuggable apps; jailbreak for unrestricted access
Read application files	P	S/P	P	S	S	S	AFC/House Arrest or app instrumentation

Capability	Stock	Dev Mode	Managed	Jailbroken	SRD	Simulator	Primary mechanism / limitation
Broader filesystem	U	U	U	S/P	S	S	Jailbreak/rootless restrictions may still protect sealed/system areas
Root filesystem	U	U	U	P/S	S	S	No ordinary retail interface
Execute generic shell	U	U	U	S	S	S	SSH/bootstrap; SRD shell; simulator <code>simctl spawn</code> /host tools
Commands as <code>mobile</code>	U	U	U	S	S	S	Jailbreak/SRD
Commands as root	U	U	U	S/P	S	S	Jailbreak implementation/ security mitigations may constrain operations
List processes	P	P/S	P	S	S	S	Developer services may expose process metadata; jailbreak gives broad access
Kill arbitrary processes	U	U/P	U	S	S	S	Stock development tools only control entitled/ debuggable targets
Inspect process memory	U	P	U/P	S	S	S	Own debuggable app via LLDB/ Frida Gadget; broad access requires privilege

Capability	Stock	Dev Mode	Managed	Jailbroken	SRD	Simulator	Primary mechanism / limitation
Dump process memory	U	P	U/P	S	S	S	Same boundary
Attach debugger	U	S/P	P	S	S	S	get-task-allow / debuggable target on stock
LLDB	U	S/P	P	S	S	S	Xcode/LLDB; arbitrary system-process access not available on stock
Syslog stream	S/P	S	S/P	S	S	S	syslog relay / OSLog / developer services
Unified logs	P	S	P/S	S	S	S	OSLog/ RemoteXPC/ pymobiledevice3; visibility varies
Crash reports	S	S	S	S	S	S	crashreport services/Xcode/ device logs
Hang reports	P	S	P/S	S	S	S	Xcode diagnostics/ MetricKit/ sysdiagnose where available
General diagnostics	S/P	S	S	S	S	S	diagnostics relay, sysdiagnose, CoreDevice, MDM
Network information	S/P	S	S	S	S	S	device services/ app diagnostics

Capability	Stock	Dev Mode	Managed	Jailbroken	SRD	Simulator	Primary mechanism / limitation
Packet capture	P	S/P	P/S	S	S	S	Remote virtual interface/ pymobiledevice3 PCAP/proxy; encryption still applies
CPU metrics	P	S	P	S	S	S	MetricKit/ Instruments/ xctrace
Memory metrics	P	S	P	S	S	S	MetricKit/ Instruments
Energy metrics	P	S	P	S/P	S/P	P	Instruments/ MetricKit; simulator cannot reproduce physical energy behaviour faithfully
CPU profiling	U/P	S	U/P	S	S	S	Instruments/ xctrace; requires development/ debug access appropriate to target
Memory profiling	U/P	S	U/P	S	S	S	Instruments/ Allocations/ Leaks etc.
Energy profiling	U/P	S	U/P	S/P	S/P	P	Instruments on physical hardware preferred
Filesystem monitoring	U	P	U/P	S	S	S	App-local instrumentation on stock; broad tracing privileged

Capability	Stock	Dev Mode	Managed	Jailbroken	SRD	Simulator	Primary mechanism / limitation
Network monitoring	P	S	P/S	S	S	S	Network instrument/RVI/PCAP/proxy
Runtime instrumentation	U/P	P/S	U/P	S	S	S	Gadget/LLDB on own/debuggable app; Frida Server on jailbreak
Objective-C method tracing	U/P	P	U/P	S	S	S	Frida/LLDB
Swift/runtime tracing	U/P	P	U/P	S	S	S	Frida/LLDB/ Instruments; Swift symbol/runtime complexity applies
Native C/C++ tracing	U/P	P	U/P	S	S	S	Frida Interceptor/ Stalker where supported, LLDB, xctrace
Syscall tracing	U	P	U	P/S	P/S	S/P	System Trace/ privileged tracers; no promise of unrestricted DTrace-like behaviour
Hook functions	U/P	P	U/P	S	S	S	Frida Gadget/ Server, debugger
Inspect arguments/ returns	U/P	P	U/P	S	S	S	Frida/LLDB
Modify runtime behaviour	U/P	P	U/P	S	S	S	Same; stock subject to code-signing/runtime restrictions

Capability	Stock	Dev Mode	Managed	Jailbroken	SRD	Simulator	Primary mechanism / limitation
Load instrumentation agent	P	S/P	P	S	S	S	Embedded/re-signed Gadget on stock; server/injection on privileged device
Frida Gadget	P	S/P	P	S	S	S	Jailed iOS has significant code-signing/hooks restrictions
Frida Server	U	U	U	S	P/S	N/A/P	Requires privileged environment; normal simulator Frida does not need iPhone-style server deployment
Interact with system daemons	U/P	U/P	U/P	S	S	P/S	Stock can call exported services; cannot arbitrarily attach/control daemons
Inspect own-app keychain behaviour	P	S	P/S	S	S	S	Test through app APIs/instrumentation
Extract arbitrary protected keychain material	U	U	U	P	P/S	Data Protection/ Secure Enclave remain important even on privileged devices	
Inspect app entitlements	S	S	S	S	S	S	codesign, Mach-O tooling, installed-artifact inspection

Capability	Stock	Dev Mode	Managed	Jailbroken	SRD	Simulator	Primary mechanism / limitation
Inspect signing information	S	S	S	S	S	S	<code>codesign</code> , provisioning inspection
Retrieve device diagnostics	S/P	S	S	S	S	S	diagnostics relay/ CoreDevice/ MDM
Modify management configuration	U/P	U/P	S	S	S	S	MDM profiles; jailbreak files/ preferences; simulator defaults/files
Modify arbitrary system configuration	U	U	U/P	S/P	S	S	MDM changes only supported policy/ configuration payloads
Automatic application-crash recovery	S	S	S	S	S	S	Lab worker/ orchestrator
Automatic device recovery	P	P	S/P	P	P	S	Reboot/Return to Service/ reconnect/ quarantine; physical failure still requires hands-on recovery

The distinction between **P** and **S** is deliberately conservative. For example, `libimobiledevice` exposes a surprisingly broad set of Apple device services without a jailbreak — app installation, crash collection, screenshots, diagnostics, syslog, file-access services, provisioning, developer-image services, WebKit debugging and more — but that is still not equivalent to arbitrary filesystem access or a shell. The project explicitly works without a jailbreak and implements Apple’s exposed device protocols independently of proprietary Apple libraries. ⁸

Similarly, `pymobiledevice3` has grown into one of the most capable non-Apple device-service toolkits, exposing AFC, application management, crash reports, OSLog/syslog, PCAP, WebInspector/CDP, recovery/DFU functionality and newer DVT/RemoteXPC developer capabilities. That makes it extremely useful as an adapter, but not proof that Apple supports those interfaces as long-term public APIs. ⁹

What supervision and MDM materially add

Supervision is valuable for a lab even though it does not solve the low-level access problem. It provides a stronger lifecycle/control channel: configuration profiles, restrictions, remote lock/erase, restart/shutdown and automated provisioning/reset workflows. Apple's Return to Service workflow can wipe and return supported devices into managed operation without the same amount of manual reconfiguration. ¹⁰

For the 2026/iOS 27 generation Apple is also adding richer declarative-management health/status information, including device system-health information. That is useful to a fleet-health controller, although it should supplement rather than replace USB-level health telemetry. ¹¹

An on-prem lab should therefore consider supervising **most organisation-owned QA devices**, but maintain the semantic distinction:

```
supervised = administratively controllable
developer_mode = development/test execution permitted
jailbroken = privileged research access
srd = Apple-sanctioned research access
```

These are four different booleans/classes.

Apple-native diagnostics and profiling

Instruments should remain a first-class backend rather than an optional developer convenience. Apple continues to expose profiling through Instruments and `xctrace`; Apple specifically documents `xctrace` as a command-line path for programmatic Instruments recordings, which is exactly what an automated investigation service needs. Current Instruments material includes Time Profiler, System Trace and modern Swift-concurrency analysis. ¹²

MetricKit belongs in the **application telemetry** layer, not the interactive device-debugging layer. It can provide application metrics and diagnostics, including crash/hang-related diagnostics, but delivery semantics differ from a live profiler; Apple documents metric payload delivery on a periodic basis and diagnostics with separate delivery behaviour. ¹³

XCTest attachments are also worth standardising. Tests can attach screenshots, files, strings and related evidence directly into results, giving deterministic test suites a native evidence format that can later be ingested into the lab artifact model. ¹⁴

ARTEMIS/Android versus iPhone

An "ARTEMIS for iPhone" cannot be a one-for-one Android port:

Android concept	Closest iOS equivalent	Important difference
ADB transport	usbmuxd + CoreDevice/ RemoteXPC	Multiple protocols/services rather than one universal interface
<code>adb devices</code>	<code>devicectl</code> , usbmuxd, libimobiledevice	Straightforward device discovery is achievable
<code>adb install</code>	devicectl / installation proxy / MDM	Signing/provisioning rules substantially stronger
UIAutomator	XCTest/XCUITest/WDA	Good UI automation, but test-host/signing infrastructure required
Logcat	OSLog/syslog/crash services	No single identical universal stream
<code>adb shell</code>	No equivalent on stock retail iOS	Fundamental capability gap
<code>adb root</code> / debug build	Jailbreak or SRD	Not a stock retail-device option
Port forwarding	usbmuxd/ <code>iproxy</code> / RemoteXPC	Achievable
App files	AFC/House Arrest	Container/service scoped
Root filesystem	None on stock	Jailbreak/SRD only
Arbitrary process attach	None on stock	Debug-entitled target only
Frida Server	Jailbreak/SRD	Stock generally uses Gadget/debuggable-app route
Recovery tooling	CoreDevice/recovery/DFU tooling	Strong device-recovery capabilities, different API model

That leads to an important product decision: the internal API should offer an **ADB-like ergonomic experience without claiming ADB-like capability parity**.

A request such as:

```
ios.shell.exec
```

must return a structured `CAPABILITY_UNAVAILABLE` on a stock iPhone rather than trying brittle workarounds.

Tooling ecosystem and project assessment

Apple, XCTest, WebDriverAgent, and Appium

XCTest is the correct supported foundation for standard-device UI automation. WebDriverAgent turns XCTest capabilities into a remotely callable WebDriver-style server. WDA supports actions such as launching/killing applications, tapping/scrolling and querying the view hierarchy, and Appium's XCTest driver builds a production automation stack around it. ¹⁵

Current Appium XCTest architecture for a physical device is approximately:

```
Appium client
|
Appium server
|
XCTest Driver
|
macOS host tooling
|
xcodebuild / preinstalled WDA
|
USB / RemoteXPC
|
WebDriverAgent XCTest runner
|
XCTest APIs
|
iOS UI / application
```

Real-device WDA still has a signing/provisioning requirement. Appium documents that valid Apple code signing is required and notes that automatic provisioning needs an Apple account/team configuration. Its documentation also warns that modern iOS signing validation can introduce online dependencies unless the chosen provisioning mechanism is suitable for offline use. ¹⁶

For a lab, **preinstalled WDA** is preferable to rebuilding it for every session. Appium documents preinstalled-WDA operation specifically as a way to avoid repeated `xcodebuild` startup overhead, with newer flows integrating RemoteXPC/CoreDevice. ¹⁷

Parallel execution is explicitly supported. Each physical-device session must have its own device identifier and isolate collision-prone resources such as `wdaLocalPort`, `mjpegServerPort` and derived-data paths. Appium also allows multiple concurrent sessions in one server process, although production isolation policy may still favour worker-level segmentation. ¹⁸

There is no credible primary-source evidence for a universal “X iPhones per Mac” Appium limit.

Appium documents how to make sessions independent; it does not publish a physical-host concurrency

ceiling. Therefore this report deliberately refuses to convert “parallel supported” into unsupported claims such as “one Mac supports 25 iPhones.” Host density must be benchmarked against your workload.

libimobiledevice versus pymobiledevice3

`libimobiledevice` should be treated as a mature base layer for nonprivileged Apple-device services. Its native protocol implementation can handle discovery, pairing-related operations, app installation/removal/listing, AFC-style file transfer, crash reports, diagnostics, screenshots, syslog and other device services without requiring jailbreaking. ⁸

Useful components include:

```
usbmuxd / libusbmuxd -> USB multiplexing / device transport
idevice_id           -> discovery
ideviceinfo          -> device metadata
ideviceinstaller     -> installation proxy
idevicesyslog        -> log relay
idevicecrashreport   -> crash retrieval
idevicescreenshot    -> screenshot service
idevicediagnosics     -> diagnostics/restart/shutdown-related service
ifuse                -> AFC/House Arrest filesystem mounting
iproxy               -> usbmuxd TCP port forwarding
```

The important limitation is that an AFC mount is **not** the iPhone root filesystem. Stock access remains constrained to the services and containers Apple exposes.

`pymobiledevice3` is more aggressive and feature-rich for modern research/dev workflows. It includes newer RemoteXPC/DVT-oriented functionality, PCAP, WebInspector and broad developer-service functionality. It is extremely attractive for an internal lab, particularly because it can fill gaps Apple’s human-oriented CLIs do not expose conveniently. Its trade-offs are GPL-3.0 licensing and greater exposure to changes in undocumented/reverse-engineered protocols. ⁹

The right design is therefore:

```
application code
|
internal typed adapter
|
+-----+
| Apple supported path   | -> devicectl/CoreDevice
| mature open path       | -> libimobiledevice
| advanced fallback path | -> pymobiledevice3
+-----+
```

Do not let application/business logic depend directly on a `pymobiledevice3` Python call whose semantics may change with an iOS release.

Project assessment

GitHub activity numbers below are snapshots around 17 September 2026 and should not be treated as permanent project attributes.

Project	Current evidence	iOS physical	Multi-device	AI/MCP	Low-level	Licence
Appium XCUITest Driver — https://github.com/appium/appium-xcuitest-driver	Active Appium-supported XCUITest driver; current major generations target Appium 3. ¹⁹	Yes	Yes	Via external MCP/agent	No	Apache-2
WebDriverAgent — https://github.com/appium/WebDriverAgent	Actively maintained Appium WDA fork, XCTest-backed WebDriver server. ²⁰	Yes	Per-device runners	No	No	BSD-fami
libimobiledevice — https://github.com/libimobiledevice/libimobiledevice	Mature cross-platform device-protocol stack; no jailbreak required. ⁸	Yes	Yes	No	Device services only	LGPL/GPL compone
pymobiledevice3 — https://github.com/doronz88/pymobiledevice3	Very feature-rich modern Python implementation, including RemoteXPC/DVT, logging, PCAP, WebInspector and recovery functions. ⁹	Yes	Yes	Ships agent-oriented integration	More than libimobiledevice, still not stock root	GPL-3.0

Project	Current evidence	iOS physical	Multi-device	AI/MCP	Low-level	Licence
Frida — https://github.com/frida/frida	Mature dynamic-instrumentation ecosystem with process/device CLI and jailed/jailbroken iOS modes. ²¹	Yes, conditional	Yes	Easy to expose	High on JB/SRD	Upstream project license review required
XcodeBuildMCP — https://github.com/getsentry/XcodeBuildMCP	Current MCP/CLI for iOS/macOS development, device tooling and stateful development workflows; roughly 6.4k stars in the researched snapshot, created 2025, active in September 2026, MIT.	Yes	Development-oriented	Native MCP	Debug/log development functions	MIT
mobile-device-mcp — https://github.com/srmorete/mobile-device-mcp	Small but relevant MCP project: iOS/Android UI actions, screenshots/UI trees, app lifecycle and multi-device concepts; on-device iOS side uses XCUITest. ²²	Yes	Yes	Native MCP	No privileged device access	MIT

Project	Current evidence	iOS physical	Multi-device	AI/MCP	Low-level	Licence
Maestro — https://github.com/mobile-dev-inc/Maestro	Very active large UI/E2E project; ~15.7k stars at snapshot, Apache-2.0, MCP support. Current README explicitly advertises physical Android devices while iOS is represented in the simulator/framework support matrix, so physical-iOS support must not be assumed from marketing history.	Revalidate	Strong execution model	Native MCP	No	Apache-2
Mobile Use — https://github.com/runablehq/mobile-use	Agent-oriented mobile project; current published scope is Android-first and iOS support is not production-ready. ²³	No production iOS	Limited	AI-native	No	MIT
Dopamine — https://github.com/opa334/Dopamine	Maintained rootless semi-untethered jailbreak with current documented version/chip ranges. ²⁴	Selected combinations	N/A	No	High	MIT

Project	Current evidence	iOS physical	Multi-device	AI/MCP	Low-level	Licence
palera1n — https://github.com/palera1n/palera1n	checkm8-based maintained iOS 15+ jailbreak for vulnerable older chips, with documented A11/passcode caveats. ²⁵	A8–A11 class where OS supports it	N/A	No	High	MIT
TrollStore — https://github.com/opa334/TrollStore	Permasigned jailed-app mechanism based on CoreTrust bugs; supports specific iOS ranges, arbitrary-entitlement/root-helper possibilities but explicitly cannot inject into system processes by itself.	Selected old versions	N/A	No	Intermediate, not full jailbreak	Upstream reports NOASSER custom

TrollStore is a good illustration of why the registry needs fine-grained capabilities rather than one `jailbroken=true/false` flag. Its own documentation says it can install permanently signed applications and preserve powerful entitlements on supported versions, while also explicitly listing things it cannot do — such as ordinary tweak injection into system processes without additional exploit primitives. Its current repository metadata reports more than 22,000 stars and no standard SPDX licence assertion, so an internal legal review is appropriate before redistributing it inside a productised platform.

Existing MCPs are components, not the answer

XcodeBuildMCP is currently one of the strongest projects for giving coding agents direct access to Xcode-oriented workflows. It provides both a CLI and MCP server and documents device code-signing requirements and stateful daemon-backed operations.

It should be **integrated** because it solves a useful development-agent problem, but it should not own:

- physical inventory,
- leases,
- fencing,

- USB fault domains,
- device quarantine,
- jailbreak capabilities,
- Frida policy,
- organisation-wide artifact retention,
- multi-rack scheduling,
- CI fairness.

`mobile-device-mcp` is closer conceptually to the desired device-control API: its design includes device enumeration, screenshots, UI trees, gestures and app commands over iOS and Android, including multi-device handling. Its small size and young maintenance footprint make it better suited as code to study or fork than as the strategic control plane. ²²

Maestro's MCP is especially attractive for an **agent that turns exploration into persistent deterministic tests**. Its README explicitly describes the pattern of an agent inspecting/tapping/asserting and then writing a reusable YAML flow. That is exactly the desired "AI discovers, deterministic system remembers" pattern. However, the current upstream README's physical-device statement is Android-specific, so physical iPhone support must be proven experimentally before making it part of this lab's real-iPhone path.

I could not establish sufficiently authoritative current evidence that a project named **TestCat**, as referenced in the assignment, is a mature iPhone autonomous-agent/device-farm system matching this scope. It should therefore remain **unverified**, rather than being promoted into the architecture from name recognition alone.

Research devices, jailbreaks, and instrumentation

Current jailbreak reality

The first policy rule should be:

**Never design the research tier around "we can jailbreak whatever iPhone/iOS we need."
Buy and freeze known-good combinations.**

Dopamine's current repository documents a rootless, semi-untethered jailbreak with materially different version ranges depending on architecture. The repository currently lists arm64e coverage through iOS 17.3.1 generally, with additional later-version coverage for selected older chip generations, including A12/A13. ²⁴

palera1n remains relevant because checkm8 is a hardware-level bootrom vulnerability affecting older chip generations. Its project documents A8–A11-class support in the iOS 15+ jailbreak context, with important A11 restrictions: on iPhone 8/8 Plus/X the passcode must be disabled under applicable jailbroken conditions, and iOS 16 introduces additional setup caveats. ²⁵

That produces a practical iPhone research matrix:

Device family	Chip	Useful public research route	Relevant OS envelope	Suitability
iPhone 6s / SE 1st gen	A9	palera1n/ checkm8 family	iOS 15-era maximums	Excellent exploit-reliability lab devices, but old OS
iPhone 7	A10	palera1n/ checkm8	iOS 15-era maximums	Same
iPhone 8 / 8 Plus / X	A11	palera1n/ checkm8	Up to their iOS 16 generation	Excellent low-level devices; passcode/security caveats materially affect test design
iPhone XS / XS Max / XR	A12	Dopamine on supported builds	Depends on exact Dopamine range and OS supported by device	Valuable newer research tier
iPhone 11 / 11 Pro / SE 2	A13	Dopamine on supported builds	Current Dopamine repository documents unusually broad later-build coverage for A13	Especially attractive research inventory
iPhone 12 family and later arm64e devices	A14+	Dopamine only where exact version is covered	General current Dopamine arm64e range should not be extrapolated beyond documented versions	Buy only against exact compatibility matrix
Current-generation devices on iOS 27	Modern chips	No blanket public-jailbreak assumption	Treat as stock unless a verified current chain exists	Standard QA/SRD only

The table is deliberately based on **chip + exact OS build**, not marketing model alone. Jailbreak eligibility must be an immutable registry fact such as:

```
research:
  jailbreak:
    name: dopamine
    version: 3.x
    root_model: rootless
    persistence: semi_untethered
    verified_boot_build: 22Hxxx
    auto_update_disabled: true
```

Do not let the MDM or lab update controller silently upgrade one of these devices.

Rootless does not mean useless

A modern rootless jailbreak is still entirely sufficient for many objectives in this assignment:

```
SSH
process enumeration/control
Frida Server
runtime injection
broad application-container access
launch/service inspection
network tooling
package bootstrap
root helper processes
system log access
security testing
```

The key distinction is that rootless jailbreaks avoid or cannot freely mutate the sealed system volume in the traditional way. That limitation does **not** prevent most runtime-security research.

TrollStore's correct role

TrollStore currently documents support for iOS 14.0 beta 2 through 16.6.1 plus selected 16.7 RC and iOS 17.0 cases; the project explicitly states that ordinary 16.7.x and 17.0.1+ are not supported by the same CoreTrust bug.

Its value to a lab is different from a jailbreak:

- persistent installation of selected applications,
- arbitrary-entitlement experimentation where the platform flaw permits it,
- root-helper possibilities on particular systems,
- JIT-related research,
- a convenient bridge for research tooling.

Its own documentation explicitly states that this alone does not provide normal system-process tweak injection/platformisation.

Therefore:

```
trollstore = capability
jailbreak = capability
frida_server = capability
root_shell = capability
```

Do not collapse them into one boolean.

Apple Security Research Device

An SRD is the best Apple-sanctioned option for genuinely low-level iOS security research. Apple states that SRDs have shell access, can run the researcher's tools, can use researcher-chosen entitlements and support kernel customisation while preserving relevant iPhone security characteristics for research. ⁷

For this architecture an SRD should be its own class:

```
device_type: apple_srd
privilege:
  shell: true
  arbitrary_entitlements: true
  kernel_research: true
  security_program_restrictions: true
```

It should **not** simply be mixed into the jailbreak pool. Programme conditions, eligible research scope and Apple-specific restrictions make it a scarce, policy-governed resource. Apple also excludes Apple Pay and third-party applications from ordinary SRD research scope, which can matter if the organisation's desired test target is not its own first-party software. ⁷

Frida on stock versus research iPhones

Frida officially documents two materially different iOS modes. On a jailbroken device, `frida-server` offers the powerful process-level model most security researchers expect, including process enumeration and system/application instrumentation. On a jailed device, Frida instead relies on a debuggable application/Gadget-style path with additional signing and debugger constraints. ²⁶

Frida Gadget is a shared library that can be embedded or injected into an application when server-style injection is not available. The current Frida documentation specifically warns that jailed iOS code-signing mode restricts instrumentation — notably `Interceptor` behaviour unless an appropriate debugger/debuggable state has been established. ²⁷

Accordingly:

Operation	Stock production app	Own development/ debuggable app	Jailbroken	SRD
<code>frida-ps</code> broad process listing	No	Limited context	Yes	Yes/expected
Frida Gadget in app	Only if packaged/ signed appropriately	Yes	Yes	Yes

Operation	Stock production app	Own development/ debuggable app	Jailbroken	SRD
Frida Server	No	No	Yes	Possible/ appropriate research tooling
Hook ObjC methods	No	Yes/conditional	Yes	Yes
Hook native functions	No	Yes/conditional	Yes	Yes
Instrument system daemon	No	No	Yes where jailbreak mitigations permit	Research-capable
Read arbitrary process memory	No	Own debug target	Yes/conditional	Research-capable
Runtime modification	No	Own debug target	Yes	Yes
Root shell	No	No	Yes	Yes

This maps cleanly to the proposed MCP:

```

stock/development:
  ios.instrument.attach    -> only if target is eligible
  ios.instrument.trace     -> only if Gadget/debug entitlement available

jailbroken:
  ios.process.list
  ios.process.inspect
  ios.memory.read
  ios.instrument.attach
  ios.instrument.trace
  ios.shell.exec

SRD:
  similar high-level API
  but implementation/policy backend may differ

```

Security-research workflow

For an authorised target:

```
reserve research device
|
verify device snapshot / jailbreak health
|
install exact app artifact
|
capture baseline process/log/network state
|
run deterministic security regression
|
optional AI exploratory interaction
|
attach Frida / LLDB under policy
|
collect method/native traces
|
exercise suspicious behaviour
|
capture crash/memory/network evidence
|
reproduce with a clean control device
|
classify app vs OS vs instrumentation effect
|
produce immutable report/evidence manifest
|
reset / sanitise / release
```

The AI should never automatically interpret “research device available” as permission to attach to every process. Authorisation must be expressed separately as target scope.

AI, MCP, CI, and autonomous operation

Why AI should sit above deterministic primitives

Recent research strongly supports a hybrid design. iOSWorld, a 2026 benchmark for native iOS agent interaction, reports substantial remaining difficulty: even the strongest tested configurations were far from reliable on all tasks, particularly complex multi-application tasks. The research also shows that combining visual and structured UI information can materially improve agent performance. ²⁸

Likewise, SWE-Bench Mobile evaluates agent/model combinations on real mobile-software tasks and finds very low end-to-end success rates relative to what a production CI system requires; the same underlying model can perform very differently depending on the agent architecture. ²⁹

Therefore the lab should have four execution modes:

Mode	Use	AI involvement
Deterministic regression	Known flows, release gates, reproducibility	None or failure summarisation only
Property/fuzz testing	Input/state-space exploration	AI can propose invariants/seeds, engine performs execution
AI exploratory testing	Unknown paths, usability/state discovery, novel bug exploration	High
AI failure investigation	Cross-device comparison, log/trace interpretation, hypothesis testing	High reasoning, targeted device actions

A stable login regression should **not** consume 200 vision-model turns every CI run. The AI should discover or repair a test, then persist a deterministic representation wherever practical.

Maestro's current design articulates essentially this useful pattern: an agent can interact with a live target and turn the resulting behaviour into a reusable deterministic YAML flow.

For physical iPhones, that deterministic representation can instead be:

- XCTest/XCUITest,
- Appium/WebDriver tests,
- organisation-specific workflow DSL,
- a generated state-machine/property test,
- or Maestro only after its exact physical-iOS support is validated.

Agent architecture

```

Goal
|
v
Planner
|
+--> Scheduler query
|
+--> Deterministic baseline tests
|
v
Explorer
|
+--> screenshot + UI tree
+--> actions through typed tools
+--> coverage/state map
|
v
Investigator

```

```

|
+--> logs/crash reports
+--> comparison devices
+--> xctrace
+--> LLDB (eligible target)
+--> Frida (eligible research target)
|
v
Hypothesis engine
|
+--> targeted reproduction
+--> counterexample/control device
|
v
Verifier
|
+--> independently confirm result
|
v
Evidence/report generator
|
v
Release device

```

The **verifier** is important: an exploratory agent should not be permitted to declare its own hypothesis proven merely because one action produced an expected screenshot.

Model classes through LiteLLM

The internal orchestration layer should address **capability classes**, not model brand names:

```

model_classes:
  LOW:
    alias: lab-low
    purposes:
      - log_classification
      - device_selection
      - simple_test_planning
      - known_error_triage

  MID:
    alias: lab-mid
    purposes:
      - UI_exploration
      - test_generation
      - screenshot_reasoning

```

```
- ordinary_failure_investigation
```

HIGH:

```
alias: lab-high
```

```
purposes:
```

- cross_device_root_cause
- complex_debugging
- trace_analysis
- security_hypothesis_generation

LiteLLM remains the organisation's gateway. The orchestrator should record the **resolved model identifier returned by the gateway** in addition to the logical class so historic runs remain auditable when aliases change.

Escalation should be evidence-driven:

```
LOW -> MID
```

```
when UI reasoning or ambiguity is required
```

```
MID -> HIGH
```

```
when:
```

```
    repeated hypotheses fail,  
    cross-version behaviour conflicts,  
    LLDB/Frida traces need interpretation,  
    security analysis is requested,  
    or confidence remains below threshold
```

Screenshot-heavy operations need vision-capable models, but **not every tool call needs vision inference**. After a UI element has a stable accessibility identifier, subsequent deterministic actions should use the identifier directly.

Proposed internal API and MCP

The underlying service should expose one semantic API and generate MCP/CLI bindings from it.

```
devices.list  
devices.query  
devices.reserve  
devices.renew  
devices.release  
devices.health  
devices.quarantine
```

```
ios.app.install  
ios.app.uninstall
```

```
ios.app.launch
ios.app.stop
ios.app.restart
ios.app.list

ios.ui.inspect
ios.ui.screenshot
ios.ui.tap
ios.ui.type
ios.ui.swipe
ios.ui.long_press
ios.ui.orientation

ios.logs.start
ios.logs.stop
ios.logs.search
ios.crashes.collect
ios.diagnostics.collect
ios.metrics.collect
ios.trace.record

ios.files.list
ios.files.read
ios.files.write

ios.process.list
ios.process.inspect
ios.process.kill
ios.memory.read

ios.debug.attach
ios.debug.command
ios.instrument.attach
ios.instrument.trace

ios.network.capture
ios.shell.exec
```

But the schema must advertise dynamic capabilities:

```
{
  "device_id": "lab-ios-0042",
  "device_class": "stock_developer",
  "capabilities": {
    "ios.ui.tap": true,
    "ios.logs.stream": true,
```



```

    "ios.debug.attach": {
      "supported": true,
      "scope": "debuggable-apps-only"
    },
    "ios.instrument.attach": {
      "supported": true,
      "scope": "eligible-app-gadget-or-debugger"
    },
    "ios.shell.exec": false,
    "ios.memory.read": {
      "supported": true,
      "scope": "debuggable-target-only"
    }
  }
}

```

A jailbroken device might return:

```

{
  "device_class": "research_jailbroken",
  "capabilities": {
    "ios.ui.tap": true,
    "ios.logs.stream": true,
    "ios.process.list": true,
    "ios.memory.read": true,
    "ios.instrument.attach": true,
    "ios.shell.exec": true
  }
}

```

This capability contract is one of the most important things to build internally.

MCP security model

MCP tools should be scoped by identity and policy, not merely by server configuration:

```

role: developer
  devices.reserve
  ios.app.*
  ios.ui.*
  ios.logs.*
  ios.crashes.collect

role: debugger
  above +

```

```

ios.debug.* on own application bundle IDs

role: security_researcher
  above +
  ios.process.*
  ios.memory.*
  ios.instrument.*
  ios.shell.exec
  only on approved research-device tags

role: lab_admin
  device lifecycle/quarantine/maintenance

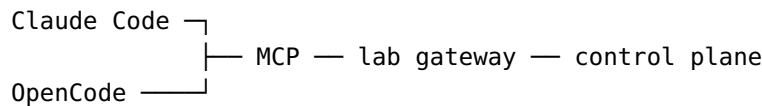
```

The LLM itself receives no authority merely because it requested a tool.

Claude Code and OpenCode

MCP is the best common denominator because Claude Code supports MCP integrations and OpenCode is intentionally model/provider-flexible. OpenCode's public design emphasises multiple model providers and local-model usage, which aligns well with an internal LiteLLM endpoint. ³⁰

The desired topology is:



not:

```
Claude Code -> SSH -> Mac mini -> arbitrary shell
```

XcodeBuildMCP can also be installed for source-tree-local developer tasks alongside the lab MCP. It currently documents MCP/CLI integration for coding agents and Xcode/device workflows.

An **adapt-before-production** generic MCP client configuration could look like:

```

{
  "mcpServers": {
    "iphone-lab": {
      "type": "http",
      "url": "https://iphone-lab.internal/mcp",
      "headers": {
        "Authorization": "Bearer ${IPHONE_LAB_TOKEN}"
      }
    }
  }
}

```

```
}  
}  
}
```

The exact Claude Code/OpenCode configuration schema should be pinned and tested in Phase 0 rather than hardcoded into the control plane, because client configuration formats evolve independently of the MCP protocol.

CI architecture

```
Git push  
|  
Build/sign  
|  
Publish immutable app artifact  
|  
Create test matrix  
|  
Scheduler resolves:  
  iPhone/model/iOS/build/device class  
  |  
Parallel leases  
  |  
Install  
  |  
Deterministic XCTest/Appium tests  
  |  
Property/fuzz suites  
  |  
AI exploratory gap coverage  
  |  
Authorised security suite  
  |  
Failure detected?  
  +----- no -----> evidence/report  
  |  
  yes  
  |  
AI investigator  
  |  
logs + crash + xctrace + comparison device  
  |  
optional LLDB/Frida  
  |  
reproduction / verification  
  |
```

```
cross-device classification
|
report
|
release
```

Build signing should occur on a controlled build/signing service rather than by distributing high-value signing credentials across every device worker.

A vendor-neutral CI job can be extremely simple:

```
set -euo pipefail

LEASE=""
cleanup() {
    if [[ -n "${LEASE}" ]]; then
        labctl devices release "${LEASE}" || true
    fi
}
trap cleanup EXIT

LEASE="$(
    labctl devices reserve \
        --where 'platform=ios AND ios>=26 AND developer_mode=true' \
        --ttl 45m \
        --output lease-id
)"

labctl ios app install \
    --lease "${LEASE}" \
    --artifact build/MyApp.ipa

labctl test run \
    --lease "${LEASE}" \
    --suite auth-regression \
    --collect logs,crashes,screenshots

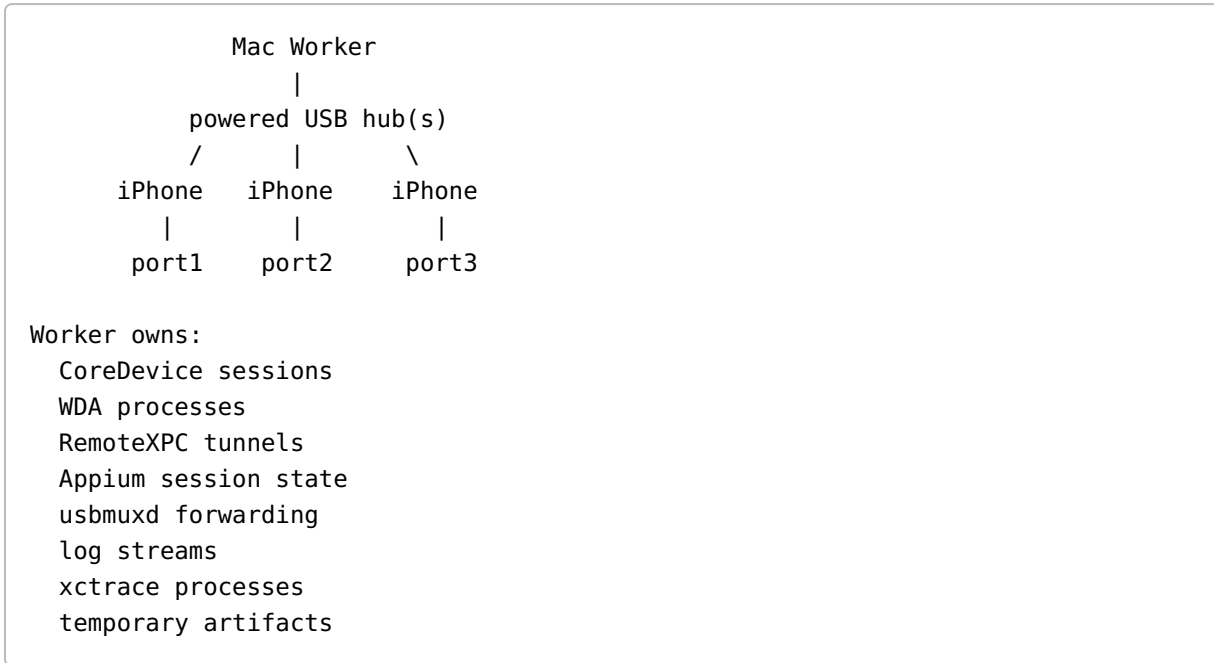
labctl investigate \
    --lease "${LEASE}" \
    --on-failure \
    --model-class MID
```

That same `labctl` contract can run under GitHub Actions, GitLab CI, Jenkins, Buildkite or an internal executor without putting CI-specific assumptions in the lab.

Device farm, scheduler, reliability, security, and scale

Worker-cell architecture

Physical devices should be grouped into relatively small **failure cells**:



A worker should be authoritative for devices physically attached to it. The central control plane should never directly manipulate a USB device.

USB as primary, network as secondary

USB should remain the preferred production path because it simultaneously provides:

- deterministic physical association,
- charging,
- low-latency transport,
- easier fault localisation,
- host ownership,
- port forwarding without dependency on lab Wi-Fi.

Wireless development should remain available for:

- network-specific test scenarios,
- resilience,
- testing actual wireless behaviour,
- cases where physical connection affects the workload.

Appium's current RemoteXPC documentation supports both usbmuxd-connected and wireless workflows and explicitly describes independent tunnels when multiple devices run concurrently. ³¹

No arbitrary devices-per-Mac number

The research did **not** find defensible primary-source evidence for statements such as:

```
1 Mac = 10 iPhones
1 Mac = 25 iPhones
1 Mac = 50 iPhones
```

Appium documents parallel isolation, not host ceilings. ¹⁸

The real limit is workload-dependent:

```
UI-only WDA session
  << resource use of
UI + MJPEG streaming
  << UI + video + continuous OSLog
  << Instruments trace
  << multiple simultaneous xcodebuild/WDA startups
```

USB topology also matters independently of CPU.

Therefore Phase 0 must certify concurrency experimentally at:

```
1 device
2 devices
4 devices
8 devices
then higher only if data justifies it
```

Measure at each level:

```
CPU p50/p95
memory pressure
swap
USB disconnect/reconnect count
WDA request p50/p95/p99
screenshot latency
MJPEG bandwidth
log throughput
xcodebuild startup time
```

```
RemoteXPC failures
trace loss
storage writes
thermal condition
session failure rate
```

A planning purchase of an eight-port industrial hub is **not evidence that eight simultaneous high-observability sessions are safe.**

Scaling model

Initial lab — 10–20 iPhones

Use multiple Mac workers rather than one huge host. Exact device density is decided by the Phase-0 benchmark. The point is to establish real USB/host failure domains from the beginning.

Medium lab — ~50 iPhones

Move to:

```
central registry
scheduler
multiple worker cells
central artifacts
NATS/Kafka-class event transport
central OTel
automatic quarantine
spare-device pool
automated USB/power recovery
```

Production — ~100 devices

Add:

```
control-plane HA
DB HA/backups
dedicated signing/build service
rack/USB topology inventory
worker autoscheduling/draining
maintenance windows
fleet-wide version policy
replacement inventory
capacity controller
```

Large lab — 100-500+

Do not build a single flat “500 phones hanging off Macs” environment. Treat racks/cells as schedulable fault domains:

```
Lab
|
+-- Rack/Cell A
|   +-- worker A1 -> phones
|   +-- worker A2 -> phones
|
+-- Rack/Cell B
|   +-- worker B1 -> phones
|   +-- worker B2 -> phones
|
+-- Research enclave
    +-- isolated workers -> JB/SRD
```

At that size the inventory system must track:

```
host controller
USB hub
hub port
cable
physical rack slot
device UDID
serial/inventory ID
model/chip
OS version/build
battery health
jailbreak state
last failure
last replacement
```

Otherwise “device disconnected” becomes an expensive physical archaeology exercise.

Device registry

Suggested object:

```
device_id: lab-ios-0042
udid: "<encrypted-or-restricted>"
platform: ios
```



```
hardware:
  marketing_model: "iPhone 15 Pro"
  model_identifier: "iPhone16,1"
  chip_family: A17Pro
  rack: rack-a
  slot: A-12

software:
  ios_version: "17.3.1"
  ios_build: "..."
  developer_mode: true
  supervised: true

research:
  jailbreak: dopamine
  jailbreak_version: "..."
  root_model: rootless
  frida: true
  frida_version: "..."

worker:
  host_id: mac-worker-07
  usb_controller: "..."
  hub_id: hub-07a
  hub_port: 4

automation:
  wda_state: healthy
  wda_bundle_id: com.org.lab.wda
  provisioning_expires_at: "..."
  remotepc: available

health:
  state: available
  battery_percent: 74
  thermal_state: nominal
  consecutive_failures: 0
  quarantine_reason: null

labels:
  - auth-test
  - no-auto-update
```

Scheduler semantics

A query such as:

```
platform = ios
model = "iPhone 15 Pro"
ios = 17.3.1
jailbroken = false
```

or:

```
platform = ios
ios >= 17
jailbroken = true
frida = true
```

should first compile into a **capability predicate**, then rank eligible devices based on health and affinity.

Reservation must be based on:

- exclusive lease by default,
- lease TTL,
- heartbeat,
- monotonic fencing token,
- priority,
- queue age,
- capability match,
- host capacity,
- maintenance state,
- quarantine state,
- affinity/anti-affinity,
- test-history isolation.

A database row lock alone is not sufficient protection against a stale worker. Every lease should have a generation/fencing token:

```
device lease generation = 918

worker command with generation 917 -> reject
worker command with generation 918 -> execute
```

This prevents an expired CI job from continuing to type into a phone that has already been reassigned.

Recovery state machine

```
action fails
|
```

```
retry idempotent operation
|
restart target app
|
restart WDA session
|
restart/prelaunch WDA
|
repair RemoteXPC tunnel
|
reconnect usbmuxd forwarding
|
reset USB port
|
reboot iPhone
|
restart worker services
|
drain/reboot Mac worker
|
power-cycle worker through PDU
|
quarantine device/cable/hub
|
human intervention
```

Do not immediately reboot a phone because a single element lookup timed out.

Failures should be classified into separate domains:

```
TEST_FAILURE
APP_FAILURE
WDA_FAILURE
DEVICE_SERVICE_FAILURE
USB_FAILURE
DEVICE_OS_FAILURE
WORKER_FAILURE
MODEL_FAILURE
CONTROL_PLANE_FAILURE
```

That classification is crucial for avoiding false product bugs.

Passcodes are a reliability constraint

Unattended post-reboot access to a passcode-protected stock iPhone is deliberately restricted. For general automated QA, the most reliable fleet policy is therefore **no passcode**, unless the test specifically requires passcode/Data-Protection behaviour.

Maintain a separate passcode-enabled sub-pool for those security scenarios.

MDM as a recovery channel

Managed devices can use Apple-supported restart/shutdown/erase commands and Return to Service where applicable. ³²

This makes MDM valuable for:

```
destructive reset
configuration enforcement
Wi-Fi/VPN/proxy profiles
update policy
device restrictions
fleet inventory/status
lost/wedged lab recovery
```

It should not be responsible for high-frequency UI test execution.

Hardware recommendation

For each cell:

```
Apple-silicon Mac worker
32+ GB RAM class for initial engineering work
ample local NVMe/SSD for Xcode/DerivedData/traces
industrial powered USB hub(s)
short labelled quality cables
per-port power/reset capability where practical
managed Ethernet
device Wi-Fi VLAN/AP
smart PDU
temperature monitoring
physical cradle/rack with airflow
```

The 32 GB/SSD guidance is an **engineering starting point**, not an Apple/Appium concurrency guarantee. Production sizing should be based on the Phase-0 profiling matrix.

Do not put every device on a cheap consumer USB hub. A hub, cable or upstream controller failure should affect the smallest possible schedulable cell.

Observability

OpenTelemetry should be the common trace model:

```
pipeline span
|
reservation span
|
device-session span
|
action span
|
adapter/tool span
|
WDA / devicectl / Frida / xctrace process
```

Every action record should include the assignment's requested fields:

```
task_id
pipeline_id
agent_id
model_class
resolved_model
device_id / restricted UDID
device_model
ios_version
ios_build
jailbreak_state
host_id
lease_generation
WDA_session
action
action_parameters_hash
screenshot_artifact
ui_tree_artifact
logs
crashes
trace IDs
Frida event stream
LLM request ID
LLM response hash
input/output tokens
```

```
latency
tool calls
errors
retries
```

A complete replay/audit manifest also needs:

```
app artifact SHA-256
test source commit
test seed
tool versions
Xcode version
WDA commit/build
Appium driver version
Frida version
device OS build
prompt/template revision
agent code revision
model alias
resolved model/version
sampling parameters
```

“Replayable” should mean **evidence-complete and reconstructable**, not an assumption that an LLM will emit byte-for-byte identical decisions.

Control-plane security

The network should be segmented into at least:

```
Control Plane VLAN
Stock Mac Worker VLAN
Stock Device Wi-Fi VLAN
Research/Jailbreak Worker VLAN
Research Device VLAN
Artifact/Observability VLAN
Model/LiteLLM VLAN
Signing/Secrets enclave
Admin network
```

The core trust assumption is:

A jailbroken iPhone is potentially hostile infrastructure.

Accordingly:

- never attach research phones to Macs containing broad signing credentials;
- never share research and normal fleet USB hubs;
- workers should initiate authenticated connections toward the control plane;
- device networks should not initiate arbitrary inbound connections to central services;
- research-worker filesystems should be disposable/reimageable;
- secrets should come from a vault/Keychain/HSM-backed service with least privilege;
- WDA signing material should be separated from high-value distribution signing material;
- `ios.shell.exec`, memory operations and Frida attach need explicit RBAC and target-scope policy;
- screenshots/logs/UI text must be treated as **untrusted model input** because a malicious application can display prompt-injection text;
- no content rendered on an iPhone should be capable of changing the agent's privileges;
- model/tool budgets and a global task kill switch are required.

On-premises and external dependencies

Most of the platform can run entirely locally after software/artifacts are mirrored:

Component	Can be on-prem?	External dependency
Scheduler/registry	Yes	None
MCP/API	Yes	None
PostgreSQL/NATS/artifacts	Yes	None
Appium/WDA runtime	Yes	Initial software/source acquisition
libimobiledevice	Yes	None after mirror
pymobiledevice3	Yes	None after mirror
Frida	Yes	None after mirror
LiteLLM/models	Yes	None if models are local
Xcode/XCTest	Mac-local	Apple distribution/licensing/update source for acquisition
WDA/dev app signing	Local once provisioned	Apple Developer programme/provisioning can introduce Apple-service dependency
MDM	Server may be on-prem	Normal Apple device management depends on Apple push/service connectivity
Device activation	Not fully eliminable in ordinary lifecycle	Apple services

Component	Can be on-prem?	External dependency
SRD	On-prem device	Apple programme relationship

This means the realistic target should be:

on-premises with tightly controlled Apple-service egress, rather than assuming a permanently air-gapped environment can preserve every stock-iPhone development/MDM workflow indefinitely.

Appium's current real-device provisioning documentation itself notes online validation considerations for modern iOS development signing. ¹⁶

Concrete implementation and rollout

Recommended repository

```

iphone-ai-lab/
├── api/
│   ├── openapi/
│   ├── protobuf/
│   └── capability-model/
├── control-plane/
│   ├── orchestrator/
│   ├── scheduler/
│   ├── registry/
│   ├── policy/
│   ├── recovery/
│   └── capacity/
├── worker-macos/
│   ├── daemon/
│   ├── device-monitor/
│   ├── session-manager/
│   ├── port-manager/
│   ├── tunnel-manager/
│   └── health/
├── adapters/
│   ├── apple-core/
│   │   ├── devicectl/
│   │   ├── xcodebuild/
│   │   ├── xctest/
│   │   ├── xctrace/
│   │   └── simctl/

```



```
|   |   └─ lldb/
|   |   └─ appium-wda/
|   |   └─ libimobiledevice/
|   |   └─ pymobiledevice3/
|   |   └─ frida/
|   |   └─ jailbreak-ssh/
|   |   └─ xcodebuildmcp/
|   |
|   └─ mcp/
|       └─ gateway/
|       └─ devices/
|       └─ ui/
|       └─ diagnostics/
|       └─ debugging/
|       └─ instrumentation/
|   |
|   └─ cli/
|       └─ labctl/
|   |
|   └─ agents/
|       └─ planner/
|       └─ explorer/
|       └─ investigator/
|       └─ security/
|       └─ verifier/
|   |
|   └─ workflows/
|       └─ deterministic/
|       └─ exploratory/
|       └─ fuzz/
|       └─ security/
|   |
|   └─ schemas/
|       └─ device/
|       └─ capability/
|       └─ lease/
|       └─ task/
|       └─ evidence/
|   |
|   └─ ci/
|       └─ github/
|       └─ gitlab/
|       └─ jenkins/
|       └─ generic/
|   |
|   └─ observability/
|       └─ otel/
|       └─ dashboards/
```

```

|   |─ alerts/
|   |─ evidence/
|
|   └─ security/
|       │─ rbac/
|       │─ policies/
|       │─ signing/
|       └─ threat-model/
|
|   └─ deployment/
|       │─ control-plane/
|       │─ macos-launchd/
|       │─ networking/
|       └─ mdm/
|
|   └─ docs/
|       │─ compatibility/
|       │─ jailbreak-matrix/
|       │─ runbooks/
|       │─ capacity/
|       └─ architecture/

```

Mac worker installation

Apple tooling should live directly on the macOS host. Containerising Xcode, CoreDevice and the physical USB ownership path is the wrong abstraction; containers are appropriate for the Linux-friendly central services.

Illustrative bootstrap, **requiring version pinning and adaptation before production**:

```

#!/bin/zsh
set -euo pipefail

# Apple toolchain must already be installed and selected.
xcode-select -p
xcodebuild -version
xcrun devicectl --help >/dev/null

# Open-source supporting tools.
brew install \
  libimobiledevice \
  libusbmuxd \
  python \
  node

# Appium 3 + XCUITest driver.

```

```
npm install -g appium
appium driver install xcuitest

# Isolate Python tooling.
python3 -m venv /opt/iphone-lab/venv
source /opt/iphone-lab/venv/bin/activate

pip install \
  pymobiledevice3 \
  frida-tools
```

Do **not** blindly replace Apple's host MobileDevice/usbmux plumbing with a third-party daemon on macOS. The worker adapter should select the appropriate tool per operation.

Discovery

Primary:

```
xcrun devicectl list devices
```

Secondary cross-check:

```
idevice_id -l
```

The worker should continuously reconcile both views into the central registry rather than trusting a one-time boot inventory. Apple officially describes `devicectl` as its command-line tool for managing/interacting with host-connected devices. ³³

Appium/WDA session configuration

Example only:

```
{
  "platformName": "iOS",
  "appium:automationName": "XCUITest",
  "appium:udid": "000081xx-...",
  "appium:bundleId": "com.example.internalapp",

  "appium:usePreinstalledWDA": true,
  "appium:updatedWDABundleId": "com.example.lab.WebDriverAgentRunner",

  "appium:wdaLocalPort": 8117,
  "appium:mjpegServerPort": 9117,
```

```
"appium:derivedDataPath": "/var/lib/iphone-lab/sessions/lease-abc123"
}
```

Unique WDA/MJPEG ports and derived-data locations are specifically recommended by Appium for parallel physical-device sessions. ¹⁸

For iOS 17+ the lab should validate a preinstalled/signed WDA strategy, and for the iOS 27 generation it must validate RemoteXPC rather than assuming the historic `devicectl` fallback is sufficient. ³⁴

WDA lifecycle state machine

```
NO_WDA
|
install signed WDA
|
INSTALLED
|
mount/verify developer services
|
establish RemoteXPC if required
|
launch WDA
|
HEALTH_CHECK
|
READY
|
session
|
failure?
+--> health probe
    -> restart runner
    -> reinstall if signature/build wrong
    -> refresh tunnel
    -> quarantine if repeated
```

A persistent worker database should track WDA build hash and provisioning expiration. Expired signing should become a **predicted maintenance event**, not a surprise CI outage.

Frida integration

Jailbroken device baseline:

```
frida-ls-devices
frida-ps -U
frida-ps -Uai
```

Target trace:

```
frida-trace \
-U \
-f com.example.internalapp \
-i 'CCCrypt*'
```

Frida's official iOS guidance supports the server model on jailbroken iOS and a different Gadget/debuggable-app approach on jailed devices. ²⁶

The worker adapter should wrap output into an event schema:

```
{
  "type": "frida.function",
  "task_id": "...",
  "device_id": "...",
  "pid": 1234,
  "module": "ExampleFramework",
  "symbol": "example_function",
  "timestamp": "...",
  "arguments": [
    {"index": 0, "representation": "..."}
  ]
}
```

Raw Frida JavaScript should be a privileged expert feature. Most autonomous tasks should invoke reviewed trace templates.

Jailbroken-device connection

Use key-only SSH and isolate transport:

```
AI/MCP
|
control plane
|
mTLS
|
research Mac worker
```

```
|  
USB/isolated research VLAN  
|  
iPhone SSH/Frida
```

Never:

```
AI agent -> direct SSH -> jailbroken phone
```

The worker should expose commands through typed RPC and policy checks.

Worker launchd service

Illustrative plist:

```
<?xml version="1.0" encoding="UTF-8"?>  
<!DOCTYPE plist PUBLIC  
    "-//Apple//DTD PLIST 1.0//EN"  
    "http://www.apple.com/DTDs/PropertyList-1.0.dtd">  
  
<plist version="1.0">  
  <dict>  
    <key>Label</key>  
    <string>com.example.iphone-lab-worker</string>  
  
    <key>ProgramArguments</key>  
    <array>  
      <string>/opt/iphone-lab/bin/lab-worker</string>  
      <string>serve</string>  
      <string>--config</string>  
      <string>/etc/iphone-lab/worker.yaml</string>  
    </array>  
  
    <key>RunAtLoad</key>  
    <true/>  
  
    <key>KeepAlive</key>  
    <true/>  
  
    <key>StandardOutPath</key>  
    <string>/var/log/iphone-lab/worker.stdout.log</string>  
  
    <key>StandardErrorPath</key>  
    <string>/var/log/iphone-lab/worker.stderr.log</string>
```

```
</dict>
</plist>
```

Production should run it under a dedicated least-privileged local account and carefully control which operations require elevation.

Worker configuration

```
worker:
  id: mac-worker-07
  pool: stock
  control_plane: https://iphone-lab.internal
  max_sessions: auto

discovery:
  devicectl: true
  libimobiledevice_crosscheck: true
  poll_interval: 5s

appium:
  endpoint: http://127.0.0.1:4723
  use_preinstalled_wda: true
  wda_port_range: [8100, 8199]
  mjpeg_port_range: [9100, 9199]

remote_xpc:
  enabled: true
  required_for_ios: ">=18"

artifacts:
  spool: /var/lib/iphone-lab/artifacts
  max_local_gb: 200

security:
  allowed_device_classes:
    - stock
    - stock_developer
  shell_exec: false
  frida_server: false
```

Research worker:

```
worker:
  id: research-mac-02
  pool: research
```

```

security:
  hostile_device_mode: true
  signing_credentials: none
  inbound_device_network: deny
  outbound_allowlist:
    - iphone-lab.internal
    - artifacts.internal

research:
  ssh: true
  frida_server: true
  privileged_operations_require_scope: true

```

Phase-zero research validation

This phase is mandatory before architecture freeze.

Validate experimentally:

Assumption	Test
iOS 27 WDA operation	Preinstall signed WDA, establish RemoteXPC, run repeated sessions/reboots
Stock-device logging	Compare devicectl, libimobiledevice and pymobiledevice3 streams
Packet capture	Validate RVI/pymobiledevice3 under intended iOS versions
Offline signing	Disconnect Apple egress and measure exactly which operations continue and for how long
Wireless operation	Pair/reconnect after worker and device restarts
MDM recovery	Restart, shutdown, lock, erase, Return to Service
WDA recovery	Kill runner/USB/tunnel and validate automated repair
Frida Gadget	Own debug-signed app, ObjC/native interception tests
Dopamine research device	Reboot/re-jailbreak/Frida/SSH reliability
palera1n device	Reboot recovery and A11/passcode constraints
MCP clients	Claude Code + OpenCode authentication/tool schema/concurrency
Concurrency	1/2/4/8 physical sessions under multiple workload profiles
USB fault isolation	Pull cables, reset ports, fail hub/worker
AI reliability	Compare deterministic baseline against agent exploration

Do not proceed to fleet procurement until the jailbreak/iOS combinations have been physically proven.
Public jailbreak compatibility tables are necessary but not sufficient for unattended lab stability.

Phase-one POC

Target:

```
1 Mac worker
2 stock Developer-Mode iPhones
1 known-good jailbroken research iPhone
simulator pool
Claude Code
OpenCode
internal LiteLLM
basic MCP/API
```

Deliver:

```
device registry
manual/short-term leases
install/launch/stop
WDA UI actions
screenshot/UI tree
logs/crashes
Frida on research phone
artifact storage
basic OTel
labctl
MCP gateway
```

Success criteria:

```
100 repeated reserve/install/test/release cycles
automatic WDA restart
automatic USB reconnect handling
deterministic evidence IDs
cross-device test execution
one end-to-end AI investigation
one authorised Frida investigation
```

Phase-two small lab

10–20 devices.

Add:

```
production scheduler
lease TTL/fencing
queues/priorities
health scores
multiple workers
automated capability discovery
CI integration
central artifacts
dashboards/alerts
quarantine
MDM lifecycle where appropriate
automated recovery ladder
```

Do not specify worker count in advance; use measured Phase-0/1 capacity.

Phase-three production lab

50–100 devices.

Add:

```
HA control plane
DB high availability/backups
worker pools by security class
dedicated signing service
multiple physical racks
per-port power control
capacity scheduling
spare inventory
advanced xctrace/Frida workflows
device lifecycle automation
strict RBAC
retention/redaction
disaster-recovery runbooks
```

Phase-four large scale

100–500+ physical devices.

Architectural changes:

cell/rack-aware scheduler
 partitioned worker pools
 capacity reservations
 global + cell-local health
 spare worker/device percentage
 fleet inventory automation
 maintenance coordinator
 rack-level failure isolation
 network QoS
 artifact lifecycle tiers
 high-volume OTel sampling
 device replacement workflow
 USB/cable component tracking

At this scale a failed \$10 cable becomes a fleet-management datum and needs its own asset identity/history.

Final stack, risks, and bibliography

Concrete final stack

Layer	Technology	Adopt / Fork / Build	Licence / model	Maturity	Main risk	Alternatives
Physical-device Apple foundation	Xcode, XCTest/ XCUITest, CoreDevice/ devicectl, xcrun	Adopt	Apple proprietary	Highest	Apple-version churn	None for supported physical iOS
UI/WebDriver	Appium 3 + XCUITest Driver	Adopt/wrap	Apache-2.0	High	WDA/ signing/ version churn	Direct XCTest
On-device UI bridge	WebDriverAgent	Adopt through Appium, prebuild/sign internally	BSD-family	High	XCTest/API breakage	Custom XCTest runner, not recommended
Apple device services	libimobiledevice	Adopt/wrap	LGPL/GPL components	High	Protocol drift	CoreDevice
Advanced device services	pymobiledevice3	Optional wrapped adapter	GPL-3.0	Strong/ community	Private-protocol churn/ licensing	CoreDevice + libimobiledevice

Layer	Technology	Adopt / Fork / Build	Licence / model	Maturity	Main risk	Alternatives
Debugging	LLDB/Xcode	Adopt	Apple/open-source components	High	Stock entitlement restrictions	Frida for eligible targets
Profiling	Instruments/xctrace	Adopt	Apple	High	Resource-heavy sessions	MetricKit for aggregate app telemetry
Security instrumentation	Frida	Adopt	Upstream component-specific	High	OS mitigations/jailbreak dependency	LLDB/custom agents
Older HW research	palera1n/checkm8	Adopt only on pinned pool	MIT	Established	Old hardware/security differences	SRD
Newer supported research	Dopamine	Adopt only on exact pinned builds	MIT	Active	Version-specific availability	SRD
Persistent jailed research apps	TrollStore	Optional	Custom/NOASSERTION upstream metadata	Established niche	Narrow version window	Development signing/jailbreak
Apple-sanctioned deep research	Security Research Device	Use if programme access available	Apple programme	Strong, scarce	Availability/scope	Jailbreak research pool
Simulator automation	XCTest/Appium; Maestro where useful	Compose	Mixed	High	Simulator $\neq$ hardware	Direct XCTest
Developer-agent helper	XcodeBuildMCP	Adopt alongside lab	MIT	Active	Not fleet manager	Internal Xcode adapter
Agent UI inspiration	mobile-device-mcp	Study/fork for POC	MIT	Early	Small maintenance base	Internal MCP over Appium

Layer	Technology	Adopt / Fork / Build	Licence / model	Maturity	Main risk	Alternatives
Deterministic/agent UI DSL	Maestro	Selective adoption	Apache-2.0	High	Physical iOS capability must be revalidated	XCTest/Appium
Fleet registry	PostgreSQL	Build schema/service	OSS	High	Schema evolution	Similar relational DB
Scheduler/leases	Internal service + PostgreSQL; NATS optional	Build	Internal/OSS	New internal	Correct fencing/recovery	Temporal etc.
Event transport	NATS JetStream or equivalent	Adopt	OSS	High	Operational complexity	Kafka/Redis Streams
Artifact storage	Internal S3-compatible object store	Adopt	Depends product	High	Volume/retention	Ceph/NAS
Observability	OpenTelemetry + internal metrics/log/trace backend	Adopt	OSS	High	Cardinality/data volume	Existing org platform
AI gateway	LiteLLM	Adopt organisation standard	Internal deployment	Existing	Model behavioural drift	Any OpenAI-compatible internal gateway
Agent interface	MCP	Adopt standard; build server	Open protocol	Rapidly maturing	Client capability differences	REST-only
Deterministic interface	labctl + REST/gRPC	Build	Internal	New	API maintenance	Direct SDK
AI orchestrator	Internal state machine	Build	Internal	New	Agent loops/cost/reliability	Workflow engines
RBAC/policy	Internal identity + policy engine	Build/integrate	Internal	Critical	Privilege escalation	Existing enterprise IAM

Build-versus-adopt verdict

Use directly: Xcode/XCTest/XCUITest, devicectl/CoreDevice, LLDB, Instruments/xctrace, Appium XCUITest, WDA, libimobiledevice, Frida, OpenTelemetry.

Use behind an adapter: pymobiledevice3, Appium, WDA lifecycle, jailbreak SSH, Frida, MDM.

Integrate but do not promote to core control plane: XcodeBuildMCP.

Evaluate/fork for ideas: mobile-device-mcp.

Use conditionally: Maestro, particularly once physical-iOS behaviour is revalidated against the current release.

Do not adopt as core iPhone technology: Android-first Mobile Use or other agents whose iOS support is a roadmap item rather than verified current functionality. ²³

Build internally: registry, capability model, scheduler, leases/fencing, recovery manager, Mac worker daemon, security policy, evidence model, unified API, unified MCP, auditability, AI orchestration.

Do not build: a new iOS UI automation driver; a new Frida, a new MobileDevice protocol implementation, or a fake stock-iOS “shell”.

The most important unresolved risks

Apple interface churn. CoreDevice/RemoteXPC behaviour has evolved materially since iOS 17, and Appium’s own 2026 documentation already contains special handling for iOS 27. The adapter layer must absorb this churn. ³⁴

Signing/provisioning dependence. WDA and development builds remain tied to Apple’s code-signing ecosystem. A completely air-gapped environment cannot be assumed to behave identically forever. ¹⁶

Jailbreak availability. No architecture should require a jailbreak on current production hardware. Dopamine and palera1n are exact hardware/OS solutions, not general iPhone features. ³⁵

Rootless/security mitigations. “Root” does not imply every kernel/system-process security mechanism disappears. Runtime tooling must report actual verified capability.

USB physical reliability. Cable/hub/controller issues can dominate at fleet scale; this must be measured rather than handled as a generic “device unavailable” error.

AI nondeterminism. Contemporary iOS/mobile-agent research is nowhere near deterministic-test reliability, particularly for complicated multi-step tasks. AI should explore/investigate, not replace the deterministic gate. ³⁶

Prompt injection from the target application. UI text, logs and web content are adversarial inputs to an autonomous agent. Privilege decisions cannot be influenced by target content.

GPL integration. pymobiledevice3 is extremely useful, but GPL-3.0 distribution/derivative-work questions should be reviewed against the organisation’s deployment model. ⁹

Commercial-device-farm host-density secrecy. Public cloud-device-farm existence proves large-scale orchestration is achievable, but public materials do not provide enough transparent host/USB topology data to use their infrastructure as evidence for “devices per Mac”. The correct answer remains local empirical certification, not a guessed number.

Final recommendation

Build the platform around **five independent capability planes**:

```
UI plane
  XCTest / WDA / Appium

Device-service plane
  CoreDevice / devicectl
  libimobiledevice / pymobiledevice3

Debug/profile plane
  LLDB / Instruments / xctrace / MetricKit

Privileged research plane
  jailbreak SSH / Frida Server / SRD

Management plane
  MDM / supervision / Return to Service
```

Put those behind:

```
Mac worker daemon
  |
capability engine
  |
scheduler / lease service
  |
policy / RBAC
  |
REST/gRPC + labctl + MCP
  |
deterministic CI + AI agents
```

This is substantially more robust than centring the design on Appium, WDA, Maestro, an MCP server, or any single mobile agent.

The core design principle is:

The unified thing should be the control plane and capability model, not the underlying iOS mechanism.

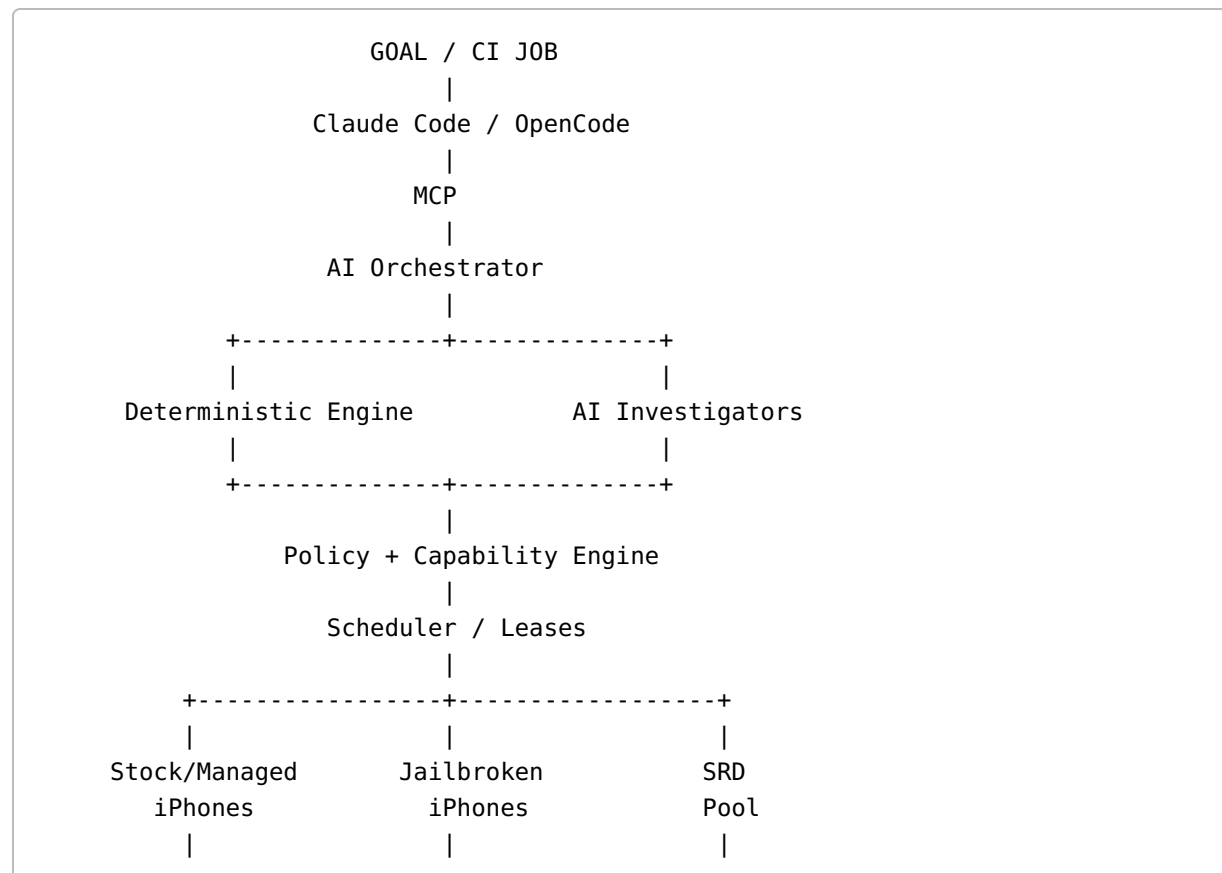
On a stock Developer-Mode iPhone the platform can provide excellent UI automation, app deployment/lifecycle control, logs, crashes, diagnostics, network capture, supported profiling and debugging of eligible application processes. XCUITest/WDA/Appium and Apple's device services are strong enough to build a serious production device farm. ³⁷

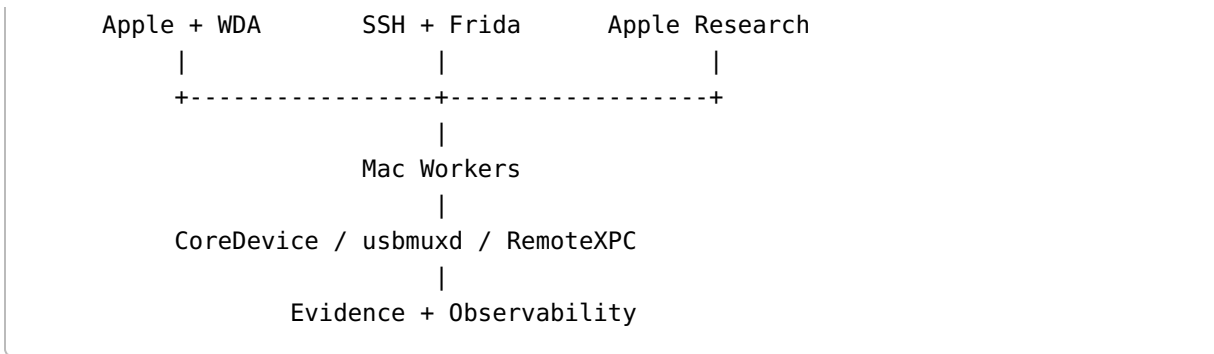
It **cannot** honestly provide arbitrary Unix shell access, root filesystem access, arbitrary system-process memory access, arbitrary hooking, or Frida Server on an ordinary retail iPhone. Those are not missing Appium features; they are consequences of the iOS security model. Frida's own documentation makes the jailed-versus-jailbroken distinction explicit. ³⁸

A deliberately frozen jailbroken pool then fills much of that gap, with A9–A11/checkm8-class devices offering highly useful older low-level research platforms and carefully selected Dopamine-compatible A12/A13/newer combinations providing more modern OS coverage. ³⁵

An Apple Security Research Device, where the organisation can obtain one and its target is within programme scope, is the strongest sanctioned complement because Apple explicitly provides shell access, custom entitlements and kernel research facilities. ⁷

The resulting platform is not “Appium at scale”. It is effectively an **iOS research operating layer**:





That architecture maximises control **without lying about stock-iOS privilege**, scales horizontally instead of depending on an undefined per-Mac device maximum, preserves deterministic testing where deterministic tools are superior, gives autonomous agents meaningful but policy-limited investigative power, supports Claude Code/OpenCode through a model-agnostic MCP boundary, and keeps the entire operational/control/model data plane on premises.

Source and repository bibliography

Apple primary material

Apple Developer documentation and WWDC material on Developer Mode, Xcode/devicectl, Instruments/xctrace, XCTest attachments, MetricKit, device supervision/MDM, Return to Service and current Xcode/iOS releases were used throughout. ³⁹

Apple Security Research Device programme documentation was the primary source for SRD capability statements. ⁷

UI automation

<https://github.com/appium/appium-xcuitest-driver> — Appium XCUITest Driver. ¹⁹

<https://github.com/appium/WebDriverAgent> — Appium WebDriverAgent. ²⁰

Current Appium documentation on parallel real-device sessions, real-device provisioning, preinstalled WDA and RemoteXPC was used for the production architecture and iOS 27 recommendations. ⁴⁰

Device protocols

<https://github.com/libimobiledevice/libimobiledevice> — open Apple device-protocol stack. ⁸

<https://github.com/doronz88/pymobiledevice3> — modern Python Apple-device tooling, RemoteXPC/DVT/WebInspector/PCAP and related functionality. ⁹

Dynamic instrumentation

<https://github.com/frida/frida> — Frida dynamic instrumentation toolkit. ⁴¹

Frida official iOS and Gadget documentation provided the jailed/jailbroken capability boundaries. ⁴²

Jailbreak/research tooling

<https://github.com/opa334/Dopamine> — current Dopamine rootless jailbreak compatibility source. ²⁴

<https://github.com/palera1n/palera1n> — current palera1n/checkm8-family research route. ²⁵

<https://github.com/opa334/TrollStore> — persistent CoreTrust-based jailed-app mechanism and entitlement/root-helper research capabilities.

AI/MCP/mobile automation

<https://github.com/getsentry/XcodeBuildMCP> — current Xcode-focused MCP and CLI, active as of September 2026, MIT.

<https://github.com/srmorete/mobile-device-mcp> — multi-device MCP-oriented iOS/Android UI-control project. ²²

<https://github.com/mobile-dev-inc/Maestro> — mature UI/E2E framework with current MCP functionality; current physical-iOS assumptions require explicit validation.

<https://github.com/runablehq/mobile-use> — Android-first mobile AI agent project; iOS not suitable as the production foundation at the researched state. ²³

The MCP specification itself provides the common agent/tool interface assumed for Claude Code/OpenCode integration. ⁴

Agent research

iOSWorld provides current empirical evidence on autonomous-agent performance in native iOS environments and the benefits/remaining limitations of vision plus structured UI representations. ²⁸

SWE-Bench Mobile provides additional evidence that model choice alone does not solve mobile-software agent reliability and that the surrounding agent architecture materially affects results. ²⁹

¹ ³⁹ <https://developer.apple.com/documentation/xcode/enabling-developer-mode-on-a-device>
<https://developer.apple.com/documentation/xcode/enabling-developer-mode-on-a-device>

² ³³ <https://developer.apple.com/documentation/xcode/xcode-command-line-tool-reference>
<https://developer.apple.com/documentation/xcode/xcode-command-line-tool-reference>

- 3 12 <https://developer.apple.com/>
<https://developer.apple.com/>
- 4 <https://modelcontextprotocol.io/>
<https://modelcontextprotocol.io/>
- 5 15 19 37 <https://github.com/appium/appium-xcuitest-driver>
<https://github.com/appium/appium-xcuitest-driver>
- 6 <https://support.apple.com/guide/deployment/about-device-supervision-dep1d89f0bff/web>
<https://support.apple.com/guide/deployment/about-device-supervision-dep1d89f0bff/web>
- 7 <https://security.apple.com/research-device/>
<https://security.apple.com/research-device/>
- 8 <https://github.com/libimobiledevice/libimobiledevice>
<https://github.com/libimobiledevice/libimobiledevice>
- 9 <https://github.com/doronz88/pymobiledevice3>
<https://github.com/doronz88/pymobiledevice3>
- 10 32 <https://support.apple.com/guide/deployment/device-management-command-list-dep789n2k1qp/web>
<https://support.apple.com/guide/deployment/device-management-command-list-dep789n2k1qp/web>
- 11 <https://support.apple.com/guide/deployment/device-management-updates-depd638aa061/web>
<https://support.apple.com/guide/deployment/device-management-updates-depd638aa061/web>
- 13 <https://developer.apple.com/documentation/metrickit/mxmetricmanager>
<https://developer.apple.com/documentation/metrickit/mxmetricmanager>
- 14 <https://developer.apple.com/documentation/xctest/adding-attachments-to-tests-activities-and-issues>
<https://developer.apple.com/documentation/xctest/adding-attachments-to-tests-activities-and-issues>
- 16 <https://appium.github.io/appium-xcuitest-driver/latest/getting-started/provisioning-profile/>
<https://appium.github.io/appium-xcuitest-driver/latest/getting-started/provisioning-profile/>
- 17 34 <https://appium.github.io/appium-xcuitest-driver/latest/guides/run-preinstalled-wda/>
<https://appium.github.io/appium-xcuitest-driver/latest/guides/run-preinstalled-wda/>
- 18 40 <https://appium.github.io/appium-xcuitest-driver/latest/guides/parallel-tests/>
<https://appium.github.io/appium-xcuitest-driver/latest/guides/parallel-tests/>
- 20 <https://github.com/appium/WebDriverAgent>
<https://github.com/appium/WebDriverAgent>
- 21 41 <https://github.com/frida/frida>
<https://github.com/frida/frida>
- 22 <https://github.com/srmorete/mobile-device-mcp>
<https://github.com/srmorete/mobile-device-mcp>
- 23 <https://github.com/runablehq/mobile-use>
<https://github.com/runablehq/mobile-use>
- 24 35 <https://github.com/opa334/Dopamine>
<https://github.com/opa334/Dopamine>

25 <https://github.com/palera1n/palera1n>

<https://github.com/palera1n/palera1n>

26 38 42 <https://frida.re/docs/ios/>

<https://frida.re/docs/ios/>

27 <https://frida.re/docs/gadget/>

<https://frida.re/docs/gadget/>

28 36 <https://arxiv.org/abs/2606.09764>

<https://arxiv.org/abs/2606.09764>

29 <https://arxiv.org/abs/2602.09540>

<https://arxiv.org/abs/2602.09540>

30 <https://opencode.ai/>

<https://opencode.ai/>

31 <https://appium.github.io/appium-xcuitest-driver/latest/guides/remotexpc-tunnels-real-devices/>

<https://appium.github.io/appium-xcuitest-driver/latest/guides/remotexpc-tunnels-real-devices/>